News Article Authentication and Fraud Detection

Aastik Biswas
BSc Data Science (Batch of 2029), St. Xavier's College (Autonomous), Kolkata

Period of Internship: 21st September 2026 – 16th October 2026

Report submitted to: IDEAS – Institute of Data Engineering, Analytics and Science Foundation, ISI Kolkata

Project Mentor: Dr. Chandan Biswas

Abstract

Fake news is common on the internet today and it is getting hard for readers to believe what they read. In this project, I built an end-to-end machine learning pipeline that can take in a news article and classify it as real or fake. The Fake and Real News dataset from Kaggle was used, with a total of 44,898 articles. The articles were cleaned, outliers in length were removed and the text was transformed into numbers using TF-IDF with 5000 features.The baseline was a Logistic Regression model which gave an F1-score of 0.989 Also experimented with K-Means clustering and TruncatedSVD and sentiment analysis to understand the structure of the data. The Random Forest with 5-fold cross-validation achieved a mean accuracy of 0.9973. The Multi-Layer Perceptron achieved an accuracy of 0.98 and an AUC of 0.9967. We also applied a custom business cost function with a penalty five times higher for a false negative than for a false positive. To summarize, the results demonstrate that fake and real articles can be separated very well by using their words.

Introduction

Most people today learn about the world through websites and social media. This makes it very easy for fake news articles to reach a large audience in a short time period, and this can damage the trust people have in real information. Due to the huge volume of content, it is impossible to check each article manually. Thus, an automatic system which can authenticate news articles is very useful. This is the main importance of this project.

The project is founded on Natural Language Processing (NLP) and supervised and unsupervised machine learning. Used Technologies: Python, pandas, NumPy, re (regular expressions), scikit-learn, TextBlob, matplotlib and seaborn. The work was done in a colab notebook. The dataset is the Fake and Real News Dataset from Kaggle, with two files, Fake.csv and True.csv, each containing a title, text, subject and date of each article.

In the background survey I read about how text classification works, the difference between bag-of-words and TF-IDF representation, how linear models, ensemble models and neural networks are used on text data. The procedure followed was: load and merge the data, explore and clean it, convert the text into TF-IDF vectors and finally train and compare different models. The goal of the project is to learn the full pipeline of a text classification problem and to test the ability to separate real and fake news.

During the first two weeks of the internship the training topics were:

- Python programming and data handling with pandas and NumPy
- Data visualization and exploratory data analysis
- Text cleaning and preprocessing with regular expressions
- Feature extraction from text (tf-idf, bag of words)
- Supervised learning: Logistic regression and random forest.
- Unsupervised learning K-Means clustering and dimension reduction
- Multi-Layer Perceptron (Neural networks) and hyperparameter tuning

- Model evaluation: f1-score, confusion matrix, roc curve and auc, cross validation.

Project Objective

The objectives of this project are as follows:

- To load the data, merge it and explore both the real and the fake news and to check if the two categories are balanced.
- We need to develop a text cleaning and preprocessing pipeline and have to transform the articles into TF-IDF features.
- We should train a baseline Logistic Regression model and then compare it with both the Random Forest and Multi-Layer Perceptron models.
- To see if fake and real news can be grouped into natural clusters using K-Means clustering and Truncated SVD.
- We should use the F1-score, the confusion matrix, ROC-AUC, and a custom business cost function to assess the models.

In this project, no hypothesis testing was carried out and no sample survey was conducted, so it is not possible to state a target population.

Methodology

The following paragraphs detail all the steps carried out in the project; since the dataset was obtained from Kaggle, no survey was carried out and no questionnaire is included.

Step 1: Data ingestion and merging

The files Fake.csv and True.csv were read into two pandas DataFrames. A new column, is_fake, was included in which the fake articles had the value 1 and the real articles had the value 0. The two DataFrames were then combined into one DataFrame named news_df and the rows were shuffled (with a random_state of 42) to ensure that the different classes are not kept together. The final size of the dataset was 44,898 rows by 5 columns.

Step 2: Exploratory data analysis

It was checked how many missing values there were in each column and it was found that the number was zero, so no rows were dropped. Next, the percentage distribution of the target variable was calculated in order to check for class imbalance.

Step 3: Text cleaning

The function clean_text() was created so as to convert the text into lowercase, remove the URLs and eliminate all punctuation marks by means of regular expressions, and it was used on the text column through the pandas apply() method with the results being saved in a new column named clean_text.

Step 4: Outlier removal using text length

The number of words in each of the cleaned articles was determined by dividing the text at whitespace intervals. It was discovered that the 99th percentile of word count was 1508 words,

and any articles with more words than this were discarded. As a result, the number of articles in the dataset was reduced from 44,898 to 44,450.

Step 5: Train-test split and TF-IDF vectorization

The clean_text column was used as the feature (X) and is_fake as the target (y). The data was divided into an 80% training set and a 20% testing set by means of train_test_split with a random_state value of 42, resulting in 35,560 articles in the training set and 8,890 in the testing set. A TfidfVectorizer that had a maximum of 5000 features was fitted only on the training data and then applied to transform both data sets, thus avoiding any data leakage.

Step 6: Model building and validation

The process of selecting a model began with a simple one and then progressed to more complex models. The models employed are listed below.

- Logistic Regression used the TF-IDF training vectors as its baseline model and had its performance evaluated on the test set using the F1-score.
- The training vectors were subjected to K-Means clustering using two clusters and then compared with the true labels by means of the Adjusted Rand Index (ARI). This serves as an unsupervised verification and is not intended for prediction.
- In order that the articles could be plotted in two dimensions, Truncated SVD was applied to reduce the 5000 TF-IDF features to 2 components.
- The sentiment polarity was calculated using TextBlob in the case of the first 1000 articles and was stored in a column called sentiment_score.
- The Random Forest (consisting of 50 trees) was validated by means of 5-fold cross-validation on the training data and the mean accuracy was then calculated.
- In order to save computation time, a Multi-Layer Perceptron (MLPClassifier) with one hidden layer containing 100 neurons, using ReLU activation and the Adam optimizer, was trained on the first 5000 training samples and tested on the first 1000 test samples.
- The learning rate and the batch size of the MLP were tuned using RandomizedSearchCV with 3 iterations and 3-fold CV on 2000 training samples.

Step 7: Error analysis and business metrics

A seaborn heatmap was used to create and display the confusion matrix for the MLP model. A custom business cost function was established with a penalty of 5 units for each false negative and 1 unit for each false positive. The ROC curve and the AUC score of the MLP were then calculated based on the predicted probabilities of the fake class.

Data Analysis and Results

A. Descriptive analysis

A summary of the dataset at various stages of the pipeline is given in Table 1.

Item	Value
Total articles after merging	44,898
Missing values	0 in all columns
Fake articles (is_fake = 1)	52.30%
Real articles (is_fake = 0)	47.70%
99th percentile of word count	1508 words
Articles after outlier removal	44,450
Training set size	35,560 articles
Testing set size	8,890 articles
TF-IDF matrix shape (training)	35,560 x 5000

Table 1: Summary of the dataset

The dataset consists of 52.30% fake articles and 47.70% real ones, which means that it is nearly balanced and there is not a serious imbalance between the classes. Accuracy is therefore a fairly reliable measure in this case, although the F1-score was also employed since it considers both precision and recall.

The sentiment polarity score was obtained for the first 1000 articles by means of TextBlob. Since the scores of the first five articles shown (0.083, 0.087, -0.005, -0.011 and 0.039) are all near zero, it follows that these articles have a fairly neutral tone. A summary of the scores for all 1000 articles was not prepared, and the sentiment score was not employed as an input to any model in this project.

A scatter plot based on the TruncatedSVD (using two components and colouring the points by is_fake) was produced in order to visually examine how well the two classes are separated in the feature space (see Figure 1). The fake articles (yellow) are mainly located in the upper section of the plot and the real articles (dark purple) are mainly in the lower section, indicating that there is a clear separation. However, there is also a substantial overlapping area in the middle, showing that the two components alone are not sufficient to fully separate the classes.

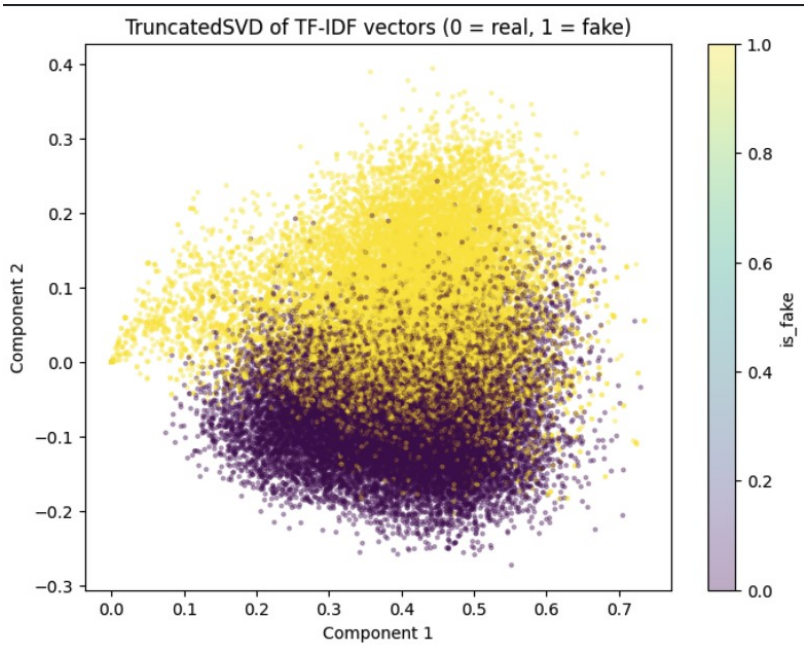


Figure 1: TruncatedSVD plot of the TF-IDF vectors (0 = real, 1 = fake)

B. Inferential analysis

Hypothesis testing was not carried out in this project. In the case of the unsupervised analysis, K-Means with two clusters produced an Adjusted Rand Index (ARI) of 0.0258. This figure is very near to zero, indicating that the clusters obtained by K-Means agree with the fake/real labels only to the same extent as a random assignment would. Therefore, the two classes are not naturally divided into two groups by the entire text and a supervised model is required in order to detect the difference.

The hyperparameter search for the MLP gave the best parameters as learning_rate_init = 0.01 and batch_size = 32. The search was run with max_iter = 20 and the optimizer showed convergence warnings, so these parameters are only indicative.

C. Model performance and comparative analysis

Model	Evaluation method	Metric	Score
Logistic Regression	Full test set (8,890 articles)	F1-score	0.9890
Random Forest (50 trees)	5-fold CV on training data	Mean accuracy	0.9973
MLP (100 neurons)	First 1000 test samples	Accuracy	0.9800
MLP (100 neurons)	First 1000 test samples	AUC	0.9967

Table 2: Comparison of the models

All three of the models worked very well; the Random Forest obtained the highest score, but its score was based on cross-validation accuracy, that of the Logistic Regression on the F1-score calculated using the entire test set, and that of the MLP on 1000 test samples after it had been trained on 5000 samples. As the criteria used and the quantities of data involved are different, the scores should not be directly compared in terms of ranking.

D. The confusion matrix and the business cost of the MLP

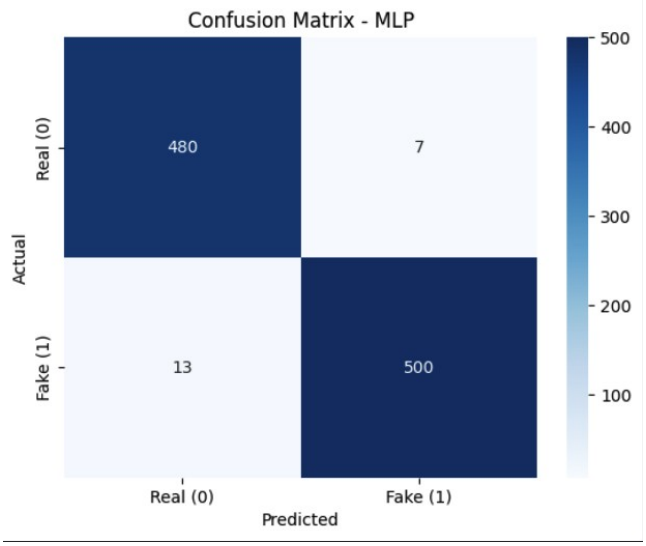


Figure 2: Confusion matrix of the MLP model on 1000 test samples

Quantity	Value
True Negatives	480
False Positives	7
False Negatives	13
True Positives	500
Total business cost (5 x FN + 1 x FP)	5 x 13 + 1 x 7 = 72

Table 3: Confusion matrix values and business cost

The MLP made 13 false negatives and 7 false positives, so it is more prone to calling fake news as real news. This is the more costly mistake as per the business cost function, and it gave a total penalty of 72 units. The ROC curve of the MLP gave an AUC of 0.9967, which is very close to 1, so the model can separate the two classes very well at different thresholds. The ROC curve is shown in Figure 3.

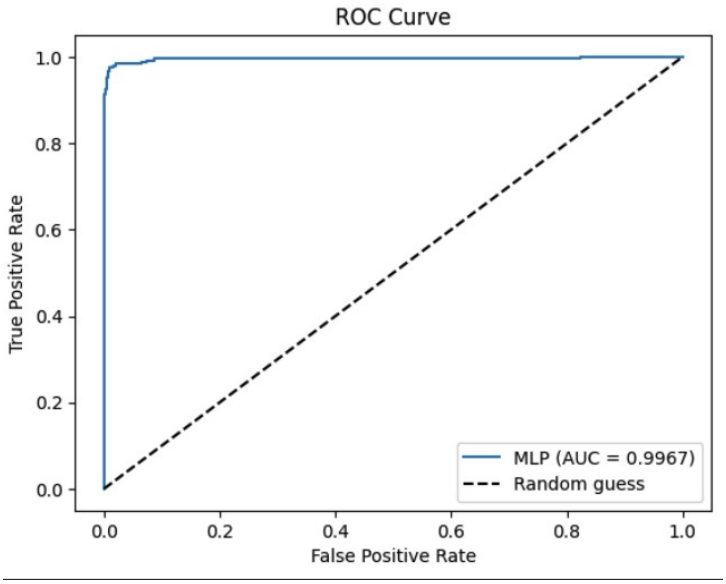


Figure 3: ROC curve of the MLP model

Conclusion

It follows from this project that it is possible to classify true and false news articles in this dataset with very high accuracy by using TF-IDF features. The baseline Logistic Regression already achieved an F1-score of 0.989, the Random Forest obtained a mean cross-validation accuracy of 0.9973, and the MLP reached an accuracy of 0.98 together with an AUC of 0.9967, even though it had been trained on only 5000 samples.

The fact that the ARI score is 0.0258 indicates that the fake and real articles do not constitute two natural groups, even though supervised models are able to separate them. This shows that the difference between the two lies in particular words which supervised models can learn, rather than in the general subject matter of the texts. The MLP had more false negatives (13) than false positives (7), and since false negatives are more serious, the business cost amounted to 72.

This study has certain limitations. The MLP was trained and tested using small subsets, and the hyperparameter search involved only 2000 samples with a small number of iterations. Moreover, the scores are very high, and this might be partly due to the way the sources in this particular dataset are written, for instance since the actual articles tend to begin with the name of a news agency, the model may not achieve the same level of performance when dealing with news from other sources.

APPENDICES

- **References**

Fake and Real News Dataset, Kaggle: https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset

scikit-learn documentation: https://scikit-learn.org/stable/

TextBlob documentation: https://textblob.readthedocs.io/

pandas documentation: https://pandas.pydata.org/docs/